

Neural Networks: From Perceptron to MLP

KAUST–Mawhiba IOAI Training

Week 2 · Day 6



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

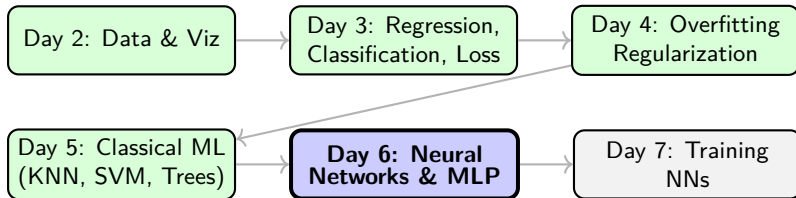
April 29, 2026

1. From Classical ML to Neural Networks
2. The Perceptron: History, Math & Limitations
3. Activation Functions: ReLU, Sigmoid, Tanh
4. Multi-Layer Perceptrons (MLP)
5. PyTorch Basics: Tensors & Operations
6. Building an MLP in PyTorch

By the end of this session, you will be able to:

1. Explain how the **Perceptron** works as a single-neuron linear classifier and identify its limitations.
2. Describe why **activation functions** (ReLU, Sigmoid, Tanh) are essential for learning non-linear patterns.
3. Build a **Multi-Layer Perceptron (MLP)** by stacking neurons into layers.
4. Create and manipulate **tensors** in PyTorch.
5. Implement a simple MLP in **PyTorch** and run a forward pass.

Recap: Our Journey So Far



Today: We move from hand-crafted ML models to **learnable, stackable building blocks** — the neuron and the neural network.

Why Do We Need Something More?

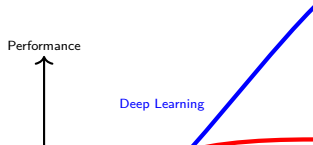
Classical ML models are powerful, but they hit a ceiling.

Classical ML Limitations

- ▶ Require **manual feature engineering**
- ▶ Scale poorly to high-dimensional data (images, audio, text)
- ▶ Fixed model structure — can't adapt complexity

Neural Networks Promise

- ▶ **Learn features automatically** from raw data
- ▶ Scale with more data & compute
- ▶ Flexible — same building blocks, any task



Traditional ML

Models are **fixed equations** designed by experts with specific assumptions about the data.

Example: Linear Regression $y = w_1x + w_0$

Benefit: Fast and interpretable.

Problem: Hard to adapt when data is complex.

Neural Networks

Models are built from **simple, stackable blocks** (like LEGO).

Example: Stacking layers



Benefit: Very flexible. Can be adapted to any data type.

Traditional ML

Models are **fixed equations** designed by experts with specific assumptions about the data.

Example: Linear Regression $y = w_1x + w_0$

Benefit: Fast and interpretable.

Problem: Hard to adapt when data is complex.

Neural Networks

Models are built from **simple, stackable blocks** (like LEGO).

Example: Stacking layers



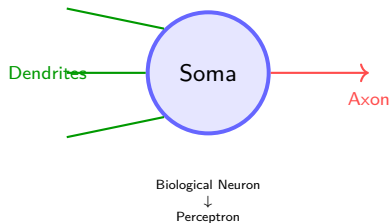
Benefit: Very flexible. Can be adapted to any data type.

*So what is this magic building block? Let's meet the **Perceptron**.*

The Perceptron (Rosenblatt, 1958)

- ▶ **Frank Rosenblatt** built the first perceptron at Cornell in 1958.
- ▶ Inspired by biological neurons in the brain.
- ▶ The simplest possible “neural network” — just **one neuron**.
- ▶ It can learn to classify data into two classes.

Key Insight: A perceptron is essentially **logistic regression** — the same model you learned on Day 3!



A perceptron performs two steps:

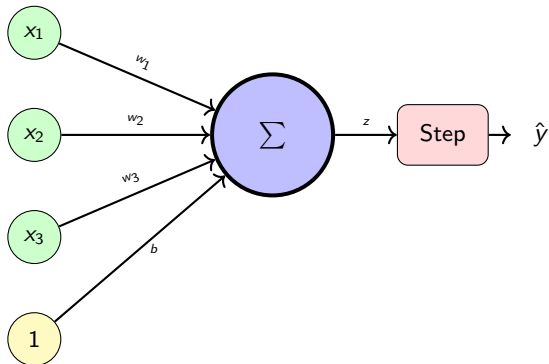
1. Weighted Sum:

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

2. Activation (Decision):

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

This is a **step function** — the simplest activation.



Remember Day 3? Logistic regression does almost the same thing:

	Perceptron	Logistic Regression
Linear part	$z = \mathbf{w}^\top \mathbf{x} + b$	$z = \mathbf{w}^\top \mathbf{x} + b$
Activation	Step function	Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
Output	$\{0, 1\}$ (hard)	$(0, 1)$ (probability)
Learning	Perceptron rule	Gradient descent

Key takeaway: A perceptron is a **single neuron** that acts as a linear classifier. When we replace the step function with sigmoid, we get logistic regression!

How does a perceptron learn? By updating weights when it makes a mistake.

Update Rule

For each training sample $(\mathbf{x}, y)$:

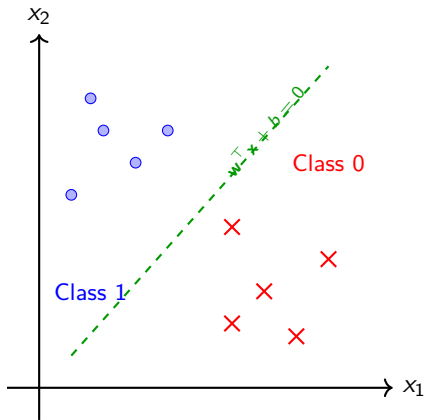
1. Compute prediction: $\hat{y} = \text{step}(\mathbf{w}^\top \mathbf{x} + b)$
2. If $\hat{y} \neq y$ (mistake), update:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha(y - \hat{y})\mathbf{x} \quad b \leftarrow b + \alpha(y - \hat{y})$$

- ▶ α is the **learning rate** (step size).
- ▶ If prediction is correct ($\hat{y} = y$), no update needed.
- ▶ **Convergence Theorem:** If the data is linearly separable, the perceptron will find a solution in finite steps.

Perceptron Decision Boundary

The perceptron draws a **linear decision boundary** to separate two classes.



The boundary is a **hyperplane**: $w_1x_1 + w_2x_2 + b = 0$.

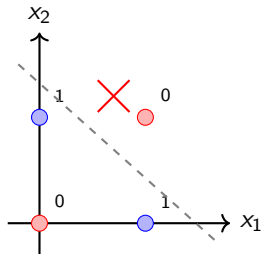
This is exactly the same as the linear decision boundary from logistic regression!

Can a perceptron learn XOR?

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

No!

XOR is **not linearly separable**. No single line can separate the two classes.



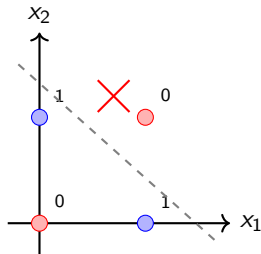
No single straight line can separate blue from red.

Can a perceptron learn XOR?

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

No!

XOR is **not linearly separable**. No single line can separate the two classes.



No single straight line can separate blue from red.

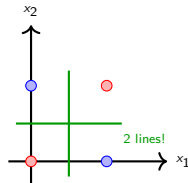
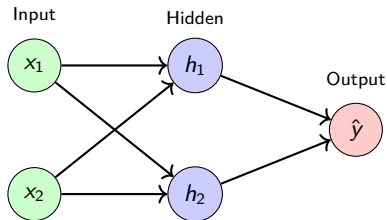
This limitation led to the “AI Winter” in the 1970s... but the solution is simple!

The Solution: Stack More Neurons!

If one neuron can only draw **one line**,
what if we use **multiple neurons**?

- ▶ Neuron 1 draws boundary #1
- ▶ Neuron 2 draws boundary #2
- ▶ A third neuron **combines** their outputs

Result: XOR solved! Two neurons in a hidden layer + one output neuron = a **Multi-Layer Perceptron (MLP)**.



What Does a Single Neuron Do?

A neuron performs two simple operations:

1. Linear Combination:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

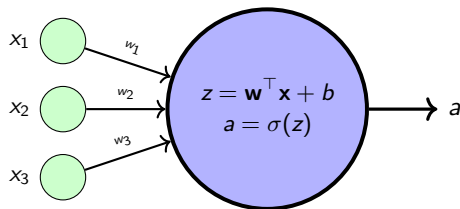
(Just like linear regression!)

2. Non-linear Activation:

$$a = \sigma(z)$$

(This is the new part!)

The output a becomes the input for the next layer.



What Does a Single Neuron Do?

A neuron performs two simple operations:

1. Linear Combination:

$$z = \mathbf{w}^T \mathbf{x} + b$$

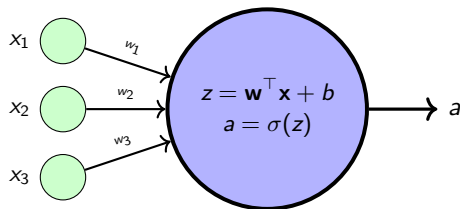
(Just like linear regression!)

2. Non-linear Activation:

$$a = \sigma(z)$$

(This is the new part!)

The output a becomes the input for the next layer.



But why do we need that activation function σ ?

Question: Why not just use the linear part ($\mathbf{w}^\top \mathbf{x} + b$)?

The Problem: Linearity Collapse

Without activations, stacking linear layers gives us... just another linear function!

$$\text{Linear} \circ \text{Linear} = \text{Linear}$$

No matter how many layers you stack, without activations, the whole network collapses into a single Linear Regression model!

Question: Why not just use the linear part ($\mathbf{w}^\top \mathbf{x} + b$)?

The Problem: Linearity Collapse

Without activations, stacking linear layers gives us... just another linear function!

$$\text{Linear} \circ \text{Linear} = \text{Linear}$$

No matter how many layers you stack, without activations, the whole network collapses into a single Linear Regression model!

The Solution — “The Bend”: Activations introduce **non-linearity**, allowing the network to “bend” and fit curved, complex patterns.

Two linear layers without activation:

$$\text{Layer 1: } z_1 = W^{(1)}\mathbf{x} + b^{(1)}$$

$$\text{Layer 2: } z_2 = W^{(2)}z_1 + b^{(2)}$$

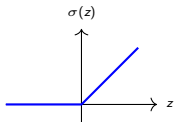
$$= W^{(2)}(W^{(1)}\mathbf{x} + b^{(1)}) + b^{(2)}$$

$$= \underbrace{W^{(2)}W^{(1)}}_{\tilde{W}}\mathbf{x} + \underbrace{W^{(2)}b^{(1)} + b^{(2)}}_{\tilde{b}}$$

This is just $\tilde{W}\mathbf{x} + \tilde{b}$ — a **single** linear transformation!
Adding more layers without activations changes nothing.

ReLU

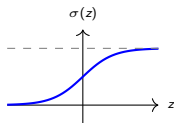
$$\sigma(z) = \max(0, z)$$



Most popular! Simple, fast, works well in deep networks.

Sigmoid

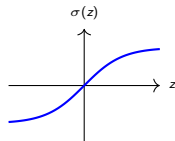
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Output in $(0, 1)$. You know this from logistic regression!

Tanh

$$\sigma(z) = \tanh(z)$$



Output in $(-1, 1)$.
Zero-centered, often better than sigmoid.

Why do these properties matter? They affect how well the network learns.

Function	Range	Zero-centered?	Derivative	Saturation?
ReLU	$[0, \infty)$	No	$\begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$	Left only
Sigmoid	$(0, 1)$	No	$\sigma(1 - \sigma)$	Both sides
Tanh	$(-1, 1)$	Yes	$1 - \tanh^2(z)$	Both sides

Saturation means the gradient becomes very small for extreme inputs $\rightarrow$ learning slows down (“vanishing gradient”). ReLU avoids this for positive inputs.

Practical Guidelines

- ▶ **Hidden layers:** Use **ReLU** (or its variants like LeakyReLU, GELU).
It's fast, avoids vanishing gradients, and works well in practice.
- ▶ **Output layer — Binary Classification:** Use **Sigmoid**.
Maps output to probability $\in (0, 1)$.
- ▶ **Output layer — Multiclass Classification:** Use **Softmax**.
Maps outputs to a probability distribution over C classes.
- ▶ **Output layer — Regression:** Use **None** (linear/identity).
Output can be any real number.

Rule of thumb: ReLU for hidden layers, task-appropriate for output.

Softmax turns a vector of raw scores into a **probability distribution**:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Example: 3-class classification

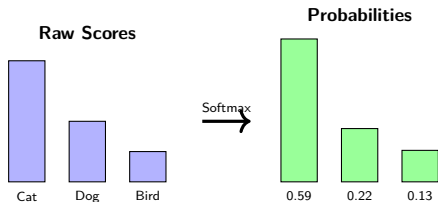
Raw scores (logits):

$$\mathbf{z} = [2.0, 1.0, 0.5]$$

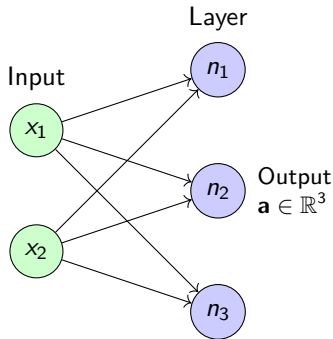
After Softmax:

$$\mathbf{p} = [0.59, 0.22, 0.13]$$

All values $\in (0, 1)$ and sum to 1.



We don't just use one neuron. We stack them in parallel to form a **Layer**.



Each neuron in the layer receives **all** inputs — this is called a **fully connected** (or **dense**) layer.

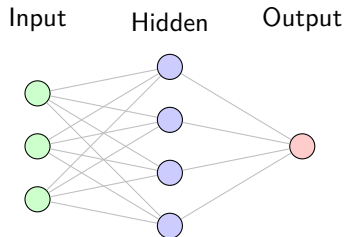
- ▶ **Input Layer:** If our data has k features (e.g., Age, Salary, Debt), the input layer has k neurons.
- ▶ **Output Layer:** Depends on the task:
 - Regression: 1 neuron (predicted value)
 - Binary classification: 1 neuron (probability)
 - Multiclass (C classes): C neurons (one score per class)
- ▶ **Hidden Layers:** This is a **design choice** (hyperparameter).
More neurons = captures more complex patterns, but risks overfitting.

A Neural Network is organized into **layers** of neurons, where each layer transforms the data step by step.

The Three Parts:

- ▶ **Input Layer:** Receives your raw data (x).
- ▶ **Hidden Layers:** Extract patterns and features.
- ▶ **Output Layer:** Produces the prediction ($\hat{y}$).

The “deep” in Deep Learning comes from having **many hidden layers**.



*Each circle is a neuron,
each line is a connection (weight).*

Since a layer is just neurons running in parallel, we can represent it using **linear algebra**.

► **Single Neuron:**

$$z = \mathbf{w}^T \mathbf{x} + b \quad (\text{weights are a vector } \mathbf{w})$$

► **Full Layer (all neurons at once):**

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad (\text{weights are a matrix } \mathbf{W})$$

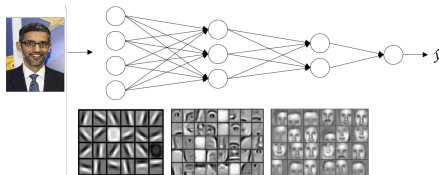
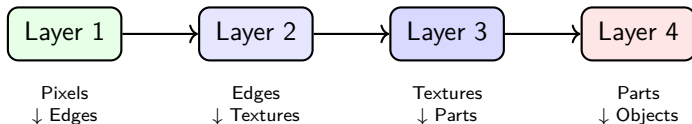
Every **row** in matrix $\mathbf{W}$ represents one neuron's weights.

If $\mathbf{W} \in \mathbb{R}^{m \times n}$: n = number of inputs, m = number of neurons in the layer.

In theory: One hidden layer is enough (Universal Approximation Theorem).

In practice: Deeper networks are much more **efficient** learners.

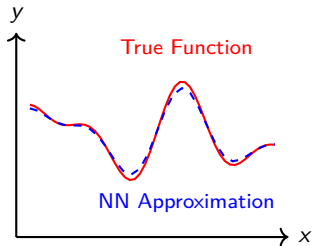
- ▶ They learn **hierarchical features**:



Universal Approximation

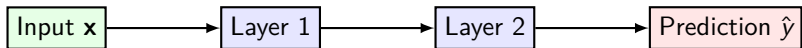
A neural network with **one hidden layer** and enough neurons can approximate **any continuous function** to arbitrary accuracy.

Caveat: “Enough neurons” could mean exponentially many. In practice, deeper (more layers) is far more efficient than wider (more neurons).



*“Give me enough neurons,
and I can fit any function.”*

A Neural Network is a **chain** of layer transformations. The output of one layer becomes the input of the next.



Mathematically (The Recursive Formula):

1. **Input:** $\mathbf{h}^{(0)} = \mathbf{x}$
2. **Hidden 1:** $\mathbf{h}^{(1)} = \sigma(\mathbf{W}_1 \mathbf{h}^{(0)} + \mathbf{b}_1)$
3. **Hidden 2:** $\mathbf{h}^{(2)} = \sigma(\mathbf{W}_2 \mathbf{h}^{(1)} + \mathbf{b}_2)$
4. **Output:** $\hat{y} = g(\mathbf{W}_3 \mathbf{h}^{(2)} + \mathbf{b}_3)$

σ : hidden activation (ReLU). g : output activation (softmax, sigmoid, or identity).

Task	Input	Output	Output Act.	Loss
Regression	$\mathbf{x} \in \mathbb{R}^k$	$\hat{y} \in \mathbb{R}$	None (linear)	MSE
Binary class.	$\mathbf{x} \in \mathbb{R}^k$	$\hat{p} \in (0, 1)$	Sigmoid	Binary CE
Multiclass	$\mathbf{x} \in \mathbb{R}^k$	$\hat{\mathbf{p}} \in \mathbb{R}^C$	Softmax	Cross-Entropy

Recipe:

Input layer $\rightarrow$ **Hidden layers** (ReLU activation) $\rightarrow$ **Output layer**
(task-specific activation)

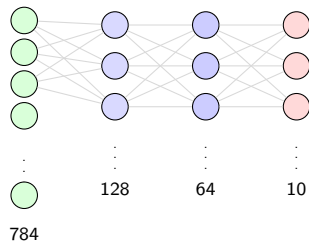
Example: MLP for Digit Classification (MNIST)

Task: Classify handwritten digits (0–9).

- ▶ Input: 28×28 image $\rightarrow$ flatten to 784 features
- ▶ Hidden Layer 1: 128 neurons + ReLU
- ▶ Hidden Layer 2: 64 neurons + ReLU
- ▶ Output Layer: 10 neurons + Softmax

Total parameters:

- ▶ $784 \times 128 + 128 = 100,480$
- ▶ $128 \times 64 + 64 = 8,256$
- ▶ $64 \times 10 + 10 = 650$
- ▶ **Total:** $\approx 109\text{K}$ parameters



Architecture (Design)

- ▶ Number of hidden layers
- ▶ Neurons per layer
- ▶ Activation functions
- ▶ Output layer shape

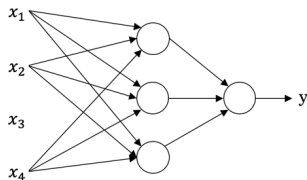
Training (Tomorrow!)

- ▶ Loss function
- ▶ Optimizer (SGD, Adam)
- ▶ Learning rate
- ▶ Batch size & epochs

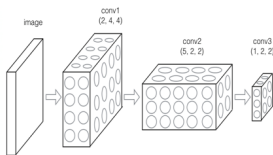
Today: We focus on architecture & building.
Day 7: We'll learn how to train
(backpropagation & gradient descent).

Types of Neural Network Architectures

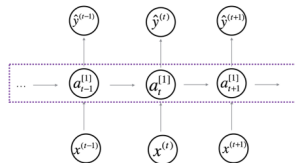
- ▶ **MLP** (Multi-Layer Perceptron): Fully connected. Best for **tabular data**. ←
Today!
- ▶ **CNN** (Convolutional Neural Network): Specialized for **images**. (Day 9)
- ▶ **RNN** (Recurrent Neural Network): Designed for **sequences** and time series.



Standard NN



Convolutional NN



Recurrent NN

What is PyTorch?

PyTorch is an open-source deep learning framework developed by Meta AI (2017).

Why PyTorch?

- ▶ **Pythonic:** Feels like writing NumPy code.
- ▶ **Dynamic computation:** Build networks on-the-fly (easy to debug).
- ▶ **GPU acceleration:** Move tensors to GPU with one line.
- ▶ **Automatic differentiation:** Computes gradients for you.
- ▶ **Industry standard:** Used by OpenAI, Tesla, NVIDIA...

PyTorch vs NumPy

NumPy	PyTorch
ndarray	Tensor
CPU only	CPU + GPU
No autograd	Autograd
No NN	torch.nn

Key idea:

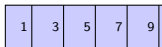
Tensors are like NumPy arrays, but with superpowers!

What is a Tensor?

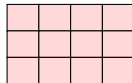
A **tensor** is a multi-dimensional array — the fundamental data structure in PyTorch.



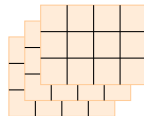
Scalar (0-D)



Vector (1-D)



Matrix (2-D)



3-D Tensor

Images are 3-D tensors (height $\times$ width $\times$ channels).
A batch of images is a 4-D tensor (batch $\times$ height $\times$ width $\times$ channels).

```
import torch

# From a Python list
x = torch.tensor([1.0, 2.0, 3.0])

# Zeros and ones
z = torch.zeros(3, 4)      # 3x4 matrix of zeros
o = torch.ones(2, 3)       # 2x3 matrix of ones

# Random values
r = torch.randn(3, 3)      # 3x3 from standard normal
u = torch.rand(2, 5)       # 2x5 from uniform [0, 1]

# Like NumPy
a = torch.arange(0, 10, 2)  # [0, 2, 4, 6, 8]
l = torch.linspace(0, 1, 5) # [0, 0.25, 0.5, 0.75, 1]

# From NumPy
import numpy as np
np_arr = np.array([1, 2, 3])
t = torch.from_numpy(np_arr) # shares memory!
```

Every tensor has three key attributes:

```
x = torch.randn(3, 4)
```

<code>x.shape</code>	<code># torch.Size([3, 4])</code>	- dimensions
<code>x.dtype</code>	<code># torch.float32</code>	- data type
<code>x.device</code>	<code># device(type='cpu')</code>	- where it lives

Common dtypes:

- ▶ `torch.float32` — default for NNs
- ▶ `torch.int64` — for labels/indices
- ▶ `torch.bool` — for masks

Moving to GPU:

```
if torch.cuda.is_available():  
    x = x.to('cuda')  
    # or: x = x.cuda()
```

One line to unlock GPU acceleration!

Element-wise operations:

```
a = torch.tensor([1., 2., 3.])
b = torch.tensor([4., 5., 6.])

a + b      # [5., 7., 9.]
a * b      # [4., 10., 18.]
a ** 2     # [1., 4., 9.]
torch.sqrt(a) # [1., 1.41, 1.73]
```

Aggregation:

```
x = torch.randn(3, 4)

x.sum()      # sum all
x.mean(dim=0) # mean per column
x.max(dim=1)  # max per row
```

Matrix operations:

```
A = torch.randn(3, 4)
B = torch.randn(4, 2)

C = A @ B      # (3, 2)
# same as:
C = torch.matmul(A, B)

# Transpose
A.T      # or A.t()      shape (4, 3)
```

@ is the matrix multiply operator
— just like NumPy!

Works exactly like NumPy:

```
x = torch.tensor([[1, 2, 3],  
                  [4, 5, 6],  
                  [7, 8, 9]])  
  
x[0]          # First row:      tensor([1, 2, 3])  
x[:, 1]       # Second col:    tensor([2, 5, 8])  
x[0:2, 1:]    # Submatrix:     tensor([[2, 3], [5, 6]])  
x[x > 5]      # Boolean mask:  tensor([6, 7, 8, 9])
```

Good news: If you know NumPy indexing, you already know PyTorch indexing!

Same syntax: [row, col], [start:end], boolean masks, fancy indexing.

Reshaping changes the tensor's shape without changing its data.

```
x = torch.arange(12)          # shape: (12,)  
  
# Reshape to matrix  
x.view(3, 4)                  # shape: (3, 4)  
x.reshape(3, 4)               # same result  
  
# -1 infers the dimension automatically  
x.view(3, -1)                 # shape: (3, 4)  -- inferred!  
x.view(-1, 6)                 # shape: (2, 6)  
  
# Add/remove dimensions  
y = torch.randn(3, 4)  
y.unsqueeze(0)                 # (1, 3, 4) -- add batch dim  
y.unsqueeze(0).squeeze(0)     # (3, 4)    -- remove it  
  
# Flatten (common before MLP layers)  
img = torch.randn(1, 28, 28)  
img.view(1, -1)               # (1, 784) -- flatten image!
```

Tip: Flattening images is how we feed them into an MLP (Day 9 will show a better way: CNNs).

Seamless conversion between NumPy and PyTorch:

```
import numpy as np

# NumPy -> PyTorch
np_array = np.array([1.0, 2.0, 3.0])
tensor = torch.from_numpy(np_array)    # shares memory!

# PyTorch -> NumPy
back_to_np = tensor.numpy()            # shares memory!

# If tensor is on GPU, move to CPU first
gpu_tensor = tensor.to('cuda')
cpu_np = gpu_tensor.cpu().numpy()
```

Warning: Shared Memory

`from_numpy()` and `.numpy()` **share memory**. Modifying one changes the other!
Use `.clone()` if you need an independent copy.

PyTorch can automatically compute **gradients** — the key to training neural networks.

```
# Create a tensor that tracks gradients
x = torch.tensor(3.0, requires_grad=True)

# Define a computation
y = x ** 2 + 2 * x + 1      # y = x^2 + 2x + 1

# Compute gradient: dy/dx = 2x + 2
y.backward()

print(x.grad)               # tensor(8.)  (2*3 + 2 = 8)
```

How it works:

1. Set `requires_grad=True` to track operations.
2. PyTorch builds a **computation graph** behind the scenes.
3. Call `.backward()` to compute all gradients via the chain rule.

PyTorch provides `torch.nn` — a library of pre-built neural network building blocks.

Key components:

- ▶ `nn.Linear(in, out)` — fully connected layer
- ▶ `nn.ReLU()` — ReLU activation
- ▶ `nn.Sigmoid()` — Sigmoid activation
- ▶ `nn.Softmax(dim)` — Softmax
- ▶ `nn.Sequential()` — stack layers

```
import torch.nn as nn

# A single linear layer
layer = nn.Linear(3, 5)
# 3 inputs, 5 outputs
# Has weights W (5x3)
# and bias b (5,)

x = torch.randn(1, 3)
out = layer(x)    # shape: (1, 5)
```

`nn.Linear(in_features, out_features)` performs $\mathbf{z} = \mathbf{x}\mathbf{W}^T + \mathbf{b}$ — the exact operation of one layer of neurons!

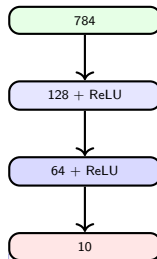
Building an MLP with nn.Sequential

The simplest way to build a network — stack layers in order:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 128),    # Layer 1: 784 -> 128
    nn.ReLU(),             # Activation
    nn.Linear(128, 64),    # Layer 2: 128 -> 64
    nn.ReLU(),             # Activation
    nn.Linear(64, 10),     # Output: 64 -> 10
)
```

```
# Forward pass
x = torch.randn(1, 784)
output = model(x)
print(output.shape)  # (1, 10)
```



For more control, define your model as a **class**:

```
class MLP(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        super().__init__()
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        x = self.layer1(x)      # Linear transformation
        x = self.relu(x)        # Non-linear activation
        x = self.layer2(x)      # Output layer
        return x

# Create model
model = MLP(input_size=784, hidden_size=128, num_classes=10)
```

__init__: Define the layers (building blocks).
forward: Define how data flows through them.

```
model = MLP(784, 128, 10)

# Print architecture
print(model)
# MLP(
#   (layer1): Linear(in_features=784, out_features=128)
#   (relu): ReLU()
#   (layer2): Linear(in_features=128, out_features=10)
# )

# Count parameters
total = sum(p.numel() for p in model.parameters())
print(f"Total parameters: {total:,}") # 101,770

# See parameter shapes
for name, param in model.named_parameters():
    print(f"{name}: {param.shape}")
# layer1.weight: torch.Size([128, 784])
# layer1.bias: torch.Size([128])
# layer2.weight: torch.Size([10, 128])
# layer2.bias: torch.Size([10])
```


Let's trace one input through the network:

```
# Single image (flattened): shape (1, 784)
x = torch.randn(1, 784)

# Step 1: Linear layer (784 -> 128)
z1 = model.layer1(x)      # shape: (1, 128)

# Step 2: ReLU activation
a1 = model.relu(z1)       # shape: (1, 128), negatives -> 0

# Step 3: Output layer (128 -> 10)
logits = model.layer2(a1) # shape: (1, 10) = raw scores

# Convert to probabilities
probs = torch.softmax(logits, dim=1)
print(probs.sum())        # tensor(1.)  -- sums to 1!

# Get prediction
pred = probs.argmax(dim=1)
print(f"Predicted class: {pred.item()}")
```

Remember Day 3? Loss functions measure how wrong our predictions are.

```
# For regression
mse_loss = nn.MSELoss()
loss = mse_loss(predictions, targets)

# For binary classification
bce_loss = nn.BCEWithLogitsLoss()
loss = bce_loss(logits, labels)    # labels: 0 or 1

# For multiclass classification (most common)
ce_loss = nn.CrossEntropyLoss()
loss = ce_loss(logits, labels)    # labels: class indices
```

Note: CrossEntropyLoss expects **raw logits** (not softmax probabilities). It applies softmax internally for numerical stability.

On Day 7, you'll implement this complete training loop:

```
model = MLP(784, 128, 10)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(num_epochs):
    for images, labels in train_loader:
        # 1. Forward pass
        predictions = model(images)
        loss = loss_fn(predictions, labels)

        # 2. Backward pass (compute gradients)
        optimizer.zero_grad()
        loss.backward()

        # 3. Update weights
        optimizer.step()

    print(f"Epoch {epoch}: Loss = {loss.item():.4f}")
```

Today: understand lines 8–9 (forward pass).

Tomorrow: understand lines 12–16 (backprop & optimization).

```
import torch
import torch.nn as nn

# 1. Define the model
class SimpleMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(4, 16), nn.ReLU(),
            nn.Linear(16, 8), nn.ReLU(),
            nn.Linear(8, 3), # 3 classes
        )
    def forward(self, x):
        return self.net(x)

# 2. Create model and dummy data
model = SimpleMLP()
x = torch.randn(5, 4) # batch of 5, 4 features each

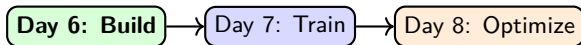
# 3. Forward pass
logits = model(x) # shape: (5, 3)
probs = torch.softmax(logits, dim=1)
preds = probs.argmax(dim=1)
print(preds) # e.g., tensor([2, 0, 1, 2, 0])
```

Theory:

1. **Perceptron:** Single neuron, linear classifier, limited by XOR.
2. **Activations:** ReLU, Sigmoid, Tanh — break linearity.
3. **MLP:** Stack layers to learn complex, non-linear patterns.
4. **Universal Approximation:** Enough neurons can fit any function.

Practical (PyTorch):

1. **Tensors:** Create, reshape, index, operate.
2. **nn.Module:** Define architectures.
3. **Forward pass:** Feed data through the network.
4. **Autograd:** PyTorch tracks gradients for you.



Tomorrow's Agenda

- ▶ **Gradient Descent:** How to find the best weights by “walking downhill” on the loss surface.
- ▶ **Backpropagation:** How PyTorch computes gradients through every layer using the chain rule.
- ▶ **SGD & Mini-Batch:** How to train efficiently with batches of data.
- ▶ **Hands-on:** Train the MLP you built today end-to-end on a real dataset!

*“A perceptron is just logistic regression.
Now watch what happens when we stack them
and teach them to learn.”*

Credits

KAUST Academy

King Abdullah University of Science and Technology

KAUST–Mawhiba IOAI 2026 Training Program